

Shared-Memory SPSC IPC and Interrupt-Mode `io_uring`: A Measurement Study of Wakeup Mechanisms and Measurement Controls

Rahul Raman Yuvraj Deshmukh B. Thangaraju

Department of Computer Science and Engineering

International Institute of Information Technology Bangalore (IIITB), Bangalore, India

rahul.raman@iiitb.ac.in yuvraj.deshmukh@iiitb.ac.in b.thangaraju@iiitb.ac.in

Abstract—Inter-Process Communication (IPC) performance is a foundational bottleneck in concurrent, multi-process software systems. Traditional kernel-mediated mechanisms—POSIX Pipes, UNIX Domain Sockets, POSIX Message Queues—impose mandatory system calls, context switches, and user-to-kernel memory copying on every message, creating an irreducible per-message overhead of several microseconds. Modern high-performance systems reduce this cost by employing lock-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffers that transfer payload data entirely in userspace via `memcpy` into POSIX shared memory regions. A critical and largely unanswered question, however, is: when the shared ring empties and the consumer must sleep, which wakeup primitive should be used—and what does Linux’s `io_uring` framework specifically contribute over simpler alternatives such as `futex` or `eventfd`?

This paper presents a rigorous empirical measurement study that decouples payload transport from wakeup-mechanism signaling and applies a two-suite evaluation framework: (1) a wakeup-mechanism ablation comparing six coordination strategies (`busy_poll`, `spin_backoff`, `adaptive`, `futex`, `eventfd`, `io_uring`) on an identical cache-aligned SPSC ring under saturated and bursty traffic conditions across a 64 B–1 MiB message-size sweep; and (2) a corrected depth-1 ping-pong latency suite in which both the send and receive timestamps are recorded on the single initiator core using `CLOCK_MONOTONIC_RAW`, eliminating cross-core TSC drift and in-flight pipelining artefacts, reporting full percentile distributions (P50, P90, P99, P99.9) over 100 000 round-trips per configuration.

Our results show that the shared-memory ring delivers the primary performance gains. Spinning variants achieve median single-trip latencies of 364–371 ns at 64 B—a 14.8× improvement over pipe or socket—and peak streaming throughput of 27.88 GiB/s at 64 KiB. In depth-1 ping-pong tests (Suite 2), `io_uring` incurs a modest per-event ring submission overhead (+1.3 μ s over `futex/eventfd` at 64 B). However, under bursty wakeup conditions (Suite 1), where wakeups occur when the ring empties, `io_uring` performs on par with `futex` and `eventfd` ($\approx$ 1.09–1.10 ms median wakeup latency with $\approx$ 0 % idle CPU consumption). The shared-memory ring, rather than the interrupt-mode wakeup primitive, delivers the primary gain. The available bursty and offered-load data are explicitly treated as “pre-frequency-pin” observations: their absolute wakeup latencies are confounded by DVFS/C-state ramp-up and therefore do not support a primitive-level ranking. We provide a frequency-pinned replication protocol and scope all completed `io_uring` conclusions to default interrupt mode (`flags=0`); `SQPOLL` is not measured.

Index Terms—`io_uring`, IPC, shared memory, SPSC ring buffer, lock-free, wakeup mechanism, latency, throughput, Linux kernel, systems performance

I. INTRODUCTION

Modern software infrastructure—spanning high-frequency trading (HFT) matching engines, real-time database kernels, media processing pipelines, and containerised microservice mesh proxies—relies heavily on efficient Inter-Process Communication. Even a modest per-message overhead of a few microseconds accumulates to tens of milliseconds when millions of messages flow per second. At 10 Mop/s, a 3 μ s per-message syscall overhead alone consumes 30 % of the CPU budget.

Linux provides a rich set of mature IPC primitives. POSIX FIFOs and UNIX domain sockets route all data through the kernel, imposing context switches and double memory copies on every message. POSIX message queues add priority ordering but retain kernel-mediated delivery semantics. Shared memory avoids the data-copy penalty by mapping identical physical pages into both producer and consumer address spaces; the kernel is contacted only for initial setup.

The cost of these kernel interactions is non-trivial and grows with message rate. A `write/read` pair on a POSIX pipe costs approximately 1–3 μ s in system-call overhead on modern Linux, independent of payload size. For a system exchanging 5 Mmsg/s, this overhead alone consumes a full CPU core. Beyond raw syscall cost, kernel buffers are allocated from a different NUMA domain relative to the application heap on multi-socket machines, adding DRAM access latency to every message. Real-time systems with sub-1 ms SLAs cannot tolerate this overhead when message rates exceed a few hundred thousand per second.

Lock-free SPSC ring buffers—popularised by the LMAX Disruptor [5] and Intel DPDK [6]—take shared memory a step further by coordinating access using only atomic load/store instructions on cache-line-aligned indices, avoiding any kernel involvement on the data path. Message delivery in the uncontended case reduces to a `memcpy` plus two atomic index

updates, costing roughly 200 ns at 64 B—an order of magnitude below any kernel-mediated mechanism.

A remaining challenge is the *empty-ring wakeup problem*: when the ring is empty, busy-spinning wastes a full CPU core; sleeping the consumer requires a kernel-level wakeup mechanism.

Linux’s `io_uring` interface (introduced in kernel 5.1 [1]) has attracted substantial interest for its promise of zero-syscall I/O. A growing body of practitioner literature presents “`io_uring`-based IPC rings” as offering superior latency and throughput over both traditional IPC and simpler shared-memory approaches. However, these claims conflate two independent design decisions: (1) the payload data path choice (shared memory vs. kernel-copy), and (2) the wakeup mechanism choice (`io_uring` vs. `futex` vs. `spin`). No prior measurement study isolates these dimensions on identical hardware with a controlled ablation.

Contributions

- 1) **Controlled wakeup ablation.** Six pluggable wakeup strategies are implemented on a bit-identical SPSC ring buffer; their impact on latency, throughput, CPU utilization, and syscall rate is measured across a 64 B–1 MiB payload sweep.
- 2) **Corrected depth-1 ping-pong protocol.** Queue depth is enforced at 1 and both round-trip endpoints are timestamped on the same core with `CLOCK_MONOTONIC_RAW`, eliminating TSC skew and pipelining artefacts.
- 3) **Tail-latency characterisation.** P50, P90, P99, and P99.9 percentiles are reported over 100 000 round-trips for 64 B and 4 KiB configurations. Completing the same capture at 64 KiB and 1 MiB is retained as a final measurement requirement.
- 4) **Explicit measurement scope.** We distinguish measured interrupt-mode results from unmeasured SQPOLL behavior and identify CPU-frequency pinning as a gate before drawing wakeup-primitive conclusions from low-load traffic.
- 5) **Reproducible artifact.** All source code, raw CSV data, and figure-generation scripts are published at https://github.com/Rahul09123/io_uring-IPC.

II. BACKGROUND & RELATED WORK

A. Traditional Kernel-Mediated IPC

POSIX Pipes (FIFOs) (Stevens and Rago [2]) provide uni-directional byte streams backed by a kernel circular buffer (default 64 KiB, tunable to 1 MiB via `fcntl(F_SETPIPE_SZ)`). Every write copies data from user to kernel space; every read copies it back. The producer and consumer alternate between running and sleeping states as the pipe buffer fills and drains, incurring scheduler-wakeup latency on each transition. At very small messages (64 B), the pipe’s fixed per-syscall entry cost dominates; at large messages (≥ 256 KiB), buffer-saturation and block-layer flushing limit throughput to ≈ 5 – 6 GiB/s regardless of CPU speed.

UNIX Domain Sockets bypass the network stack but retain the socket buffer abstraction (`sk_buff`). Each `sendmsg/recvmmsg` pair requires a system call and a socket-layer serialisation lock (`mutex_lock` visible in flamegraph traces). Socket send/receive buffers are tuned to 2 MiB via `SO_SNDBUF/SO_RCVBUF` in our implementation. Despite the per-call overhead, UNIX sockets achieve competitive throughput at large payload sizes because the kernel socket buffer is not bounded by a fixed slot count and can leverage zero-copy page-flipping for contiguous large writes.

POSIX Message Queues (mqueue) deliver structured, priority-sorted messages via a virtual filesystem inode under `/dev/mqueue`. A key optimisation—`pipelined_send`—delivers directly into a blocked receiver’s address space when one is already waiting, bypassing intermediate queue staging and making `mqueue` competitive with shared-memory rings at small message sizes. However, the default maximum message size is 8 KiB and the maximum queue depth is 10 messages, both controlled by kernel tunables (`fs.mqueue.msgsize_max`, `fs.mqueue.msg_max`), imposing hard architectural limits at scale.

TABLE I
Kernel-IPC System-Call and Copy Costs per Message

Mechanism	Syscalls /msg	Copies /msg	Kernel buffer
POSIX Pipe	2	2	Ring buf (64 KiB–1 MiB)
UNIX Socket	2	2	<code>sk_buff</code> (up to 2 MiB)
POSIX MQ	2	1–2	VFS inode (10 msgs)
SHM Ring	0	1	None (userspace)

B. Lock-Free SPSC Shared-Memory Rings

Shared-memory IPC maps identical physical pages into producer and consumer address spaces via `shm_open` and `mmap`. SPSC ring buffers (Lamport [4], Herlihy and Shavit [9], LMAX Disruptor [5], DPDK `rte_ring` [6]) coordinate using a pair of atomic integer indices. The critical correctness rule is: the producer advances `head` with `release` ordering after writing payload; the consumer reads `head` with `acquire` ordering before consuming.

The C++17 memory model formally defines these orderings. A *release store* ensures all prior writes are visible to any thread that subsequently performs an *acquire load* of the same variable. For an SPSC ring, this means the consumer observing `head > tail` is guaranteed to see the complete payload written by the producer before `head` was advanced. Using `memory_order_relaxed` on `head` (a common optimisation) works on x86 due to Total Store Order (TSO) but is technically undefined behaviour on weakly-ordered architectures (ARM, RISC-V, Power) that permit Store-Load reordering. We use `memory_order_seq_cst` on the publish store to maintain portability and prevent the lost-wakeup race.

False sharing—where producer and consumer cache lines collide—is prevented by placing `head`, `tail`, and control flags

on separate 64-byte-aligned cache lines using `alignas(64)`. Without this separation, a producer store to `head` invalidates the cache line containing `tail` on the consumer core, forcing a cache-coherency RFO (Request For Ownership) on every slot exchange and doubling effective memory latency.

C. The `io_uring` Interface

Introduced by Jens Axboe [1] and merged in Linux 5.1, `io_uring` exposes two ring buffers shared between userspace and the kernel: the Submission Queue (SQ) and the Completion Queue (CQ). In the default interrupt-driven mode (`flags=0`), a single `io_uring_enter` system call submits all pending SQEs and optionally waits for completions. In SQPOLL mode (`IORING_SETUP_SQPOLL`), a dedicated kernel poller thread eliminates the syscall entirely but requires elevated privileges.

In our implementation, `io_uring` is used *exclusively for the wakeup signaling path*. When the consumer sleeps, it submits an `IORING_OP_READ` on a named FIFO and blocks in `io_uring_submit_and_wait`. When the producer detects `consumer_sleeping = 1`, it submits an `IORING_OP_WRITE` to the same FIFO. The payload data path is pure `memcpy` into shared memory—`io_uring` never touches message bytes.

D. Prior Work on `io_uring` Performance

Didona et al. [7] compared `libaio`, `SPDK`, and `io_uring` for storage I/O, demonstrating that polling makes `io_uring` competitive with `SPDK` for sequential workloads. Ren and Trivedi [8] demonstrated that `io_uring` can match kernel-bypass performance for certain NVMe access patterns. Neither study addresses the question of wakeup-primitive selection for shared-memory IPC, leaving this dimension unexplored. Our work fills this gap with a controlled ablation study on a single fixed ring design.

III. SYSTEM DESIGN

Our architecture cleanly separates the data path from the control path.

A. Lock-Free SPSC Ring Buffer Layout

The shared ring buffer is a single POSIX shared memory object opened via `shm_open` and mapped with `mmap` into both the producer and consumer address spaces.

Listing 1. Cache-aligned SPSC RingBuffer (`common.h`)

```
struct alignas(64) RingBuffer {
    alignas(64) atomic<uint64_t> head;    // Producer write
    index
    alignas(64) atomic<uint64_t> tail;    // Consumer read
    index
    alignas(64) atomic<uint32_t> consumer_sleeping;

    struct alignas(64) Slot {
        uint64_t send_ns;                // Producer timestamp
        (CLOCK_MONOTONIC_RAW)
        uint64_t wakeup_publish_ns;      // Wakeup-signal timestamp
        uint32_t size;                   // Payload bytes written
        char data[MAX_PAYLOAD];          // Up to 1 MiB of payload
    };
    Slot slots[NUM_SLOTS];               // Fixed 64-slot ring
    (~64 MiB total)
};
```

Three design properties are critical.

(1) Cache-line isolation. `head`, `tail`, and `consumer_sleeping` each occupy their own 64-byte cache line. Without this separation, a producer write to `head` invalidates the cache line containing `tail` on the consumer core, forcing an RFO (Request For Ownership) round-trip on every slot transition. Our hardware counter data (Table X) confirms this: the SHM Ring’s 4.97% LLC miss rate compares favourably to 30–34% for mechanisms that traverse kernel buffers and inadvertently evict ring control variables.

(2) Sequential consistency at publish boundaries. `head` is stored with `memory_order_seq_cst` to prevent Store-Load reordering that could cause the producer to read stale `consumer_sleeping = 0` and skip sending a wakeup, silently deadlocking the consumer. On x86 (TSO), this is equivalent to a plain store followed by a `mfence`; on ARM64 it compiles to a `stlr` (store-release) instruction.

(3) Lost-wakeup prevention. A classic race exists in any producer-consumer pair: the consumer checks `head == tail` (empty), decides to sleep, but before setting `consumer_sleeping`, the producer writes a slot and checks `consumer_sleeping = 0` (decides not to signal), then the consumer sets `consumer_sleeping = 1` and waits—forever. We eliminate this race by having the consumer set `consumer_sleeping = 1` *first*, then re-check `head != tail` under a full memory fence. If a slot arrived in the window, the re-check catches it and the consumer proceeds without sleeping.

Data-flow walkthrough. On every message transfer: (a) the producer calls `memcpy` into `slots[head % NUM_SLOTS].data`, writes the timestamp and size, then executes a `seq_cst` store to `head`; (b) the consumer spin-reads `head` with acquire ordering; once it observes `head > tail`, it processes the slot and advances `tail` with a release store; (c) if the consumer finds the ring empty after its deadlock-prevention recheck, it invokes the configured wakeup primitive to sleep. Steps (a) and (b) involve zero system calls; step (c) invokes at most one system call per wakeup event.

B. Pluggable Wakeup Layer: Six Ablation Variants

To isolate the contribution of the wakeup mechanism, we implement six interchangeable wakeup modules on a single, bit-identical SPSC ring buffer data structure. Each primitive is invoked via its standard Linux kernel system interface (Linux man-pages [10]): Only the functions `consumer_wait()` and `producer_signal()` differ, guaranteeing that any observed latency difference is attributable solely to the wakeup mechanism.

a) *1. busy_poll.*: The consumer spins in a tight while(`head.load(relaxed) == tail`) loop with no backoff. Zero system calls are issued. Latency is minimal (≈ 200 ns) but one CPU core is permanently pegged at 100%.

b) *2. spin_backoff.*: Same tight spin with an `_mm_pause()` intrinsic on every iteration. `_mm_pause` inserts a brief pipeline delay on x86, reducing memory-ordering

storms and lowering power by $\approx 20\%$ vs. `busy_poll` with negligible latency impact.

c) 3. *adaptive*.: The consumer spins for up to 500 iterations; if no data arrives, it falls back to a `futex` sleep. This hybrid targets low-latency workloads where the ring is rarely empty (spin wins) while gracefully handling burst-idle traffic (`futex` sleep saves power).

d) 4. *futex*.: The consumer stores 1 to `consumer_sleeping` and calls `SYS_futex(FUTEX_WAIT)`. The kernel checks the address atomically; if it still reads 1, the thread is suspended. The producer stores 0 and calls `FUTEX_WAKE`. One system call per sleep and per wakeup event.

e) 5. *eventfd*.: The consumer polls an `eventfd` file descriptor and then reads to consume the wakeup token. The producer writes 1 to the `eventfd`. The `eventfd` is compatible with `epoll/select`, easing integration into existing event-loop frameworks.

f) 6. *io_uring*.: The consumer submits an `IORING_OP_READ_SQE` targeting a named FIFO (`/tmp/uring_sig_fifo`) and blocks in `io_uring_submit_and_wait(ring, 1)`. The producer submits an `IORING_OP_WRITE_SQE` to the same FIFO and calls `io_uring_submit`. The `io_uring` context can simultaneously monitor additional FDs, making it advantageous for frameworks managing many concurrent channels. Our primary ablation evaluates standard interrupt mode (`flags=0`); additionally, the implementation provides runtime support for `IORING_SETUP_SQPOLL` (`SQPOLL` mode via `USE_SQPOLL=1`), which offloads ring polling to an asynchronous kernel thread under `CAP_SYS_ADMIN`.

TABLE II
Wakeup Mechanism Properties Summary

Variant	Wait Strategy	Idle CPU	Sys/wake
<code>busy_poll</code>	Tight spin	100 %	0
<code>spin_back</code> .	Spin+PAUSE	High	0
<code>adaptive</code>	Spin-then-futex	Low	0–1
<code>futex</code>	<code>FUTEX_WAIT</code>	$\approx 0\%$	2
<code>eventfd</code>	<code>poll+read</code>	$\approx 0\%$	2
<code>io_uring</code>	<code>sub_and_wait</code>	$\approx 0\%$	2

IV. EXPERIMENTAL METHODOLOGY

A. Two-Suite Benchmark Framework

Suite 1 — Wakeup-Mechanism Ablation. The producer streams messages continuously under two regimes: *saturated* (back-to-back sends; ring rarely empties) and *bursty* (64-message burst, then 1 ms sleep; ring empties every burst). The bursty regime directly exposes wakeup primitive latency. Payload sizes sweep across {64 B, 256 B, 1 KiB, 4 KiB, 64 KiB, 1 MiB}. $N = 15$ measured runs per configuration (one warmup discarded).

Suite 2 — Corrected Depth-1 Ping-Pong. Queue depth is strictly enforced to 1: the initiator sends one message and blocks waiting for the echo server to reflect it before sending the next. Both t_{start} and t_{end} are recorded on **Initiator Core 1** using

`clock_gettime(CLOCK_MONOTONIC_RAW)`. Single-trip latency:

$$L_{\text{trip}} = \frac{t_{\text{end}} - t_{\text{start}}}{2} \quad [\mu\text{s}]$$

100 000 round-trips per configuration; 1 000 warmup trips discarded. P50, P90, P99, and P99.9 percentiles reported.

B. Hardware and System Configuration

TABLE III
Reference Test Environment

Parameter	Value
System	ASUS Vivobook K3605ZF
OS	Ubuntu 24.04.4 LTS (kernel 6.x)
CPU	Intel Core i5-12500H (Alder Lake, 12th Gen)
Architecture	Hybrid P/E-core
Cores/Threads	12 Physical / 16 Logical
Frequency	400–4500 MHz (boost)
L1d Cache	448 KiB (12 instances)
L2 Cache	9 MiB (6 instances)
L3 Cache	18 MiB (shared)
RAM	16 GiB DDR4-3200
Storage	512 GB Samsung NVMe SSD
Compiler	g++ -O2, C++17
liburing	System package, Ubuntu 24.04
CPU Governor	<code>schedutil/DVFS</code> in reported data; pinned replication pending

Core affinity. Producer/Initiator pinned to Core 1; Consumer/Echo-Server pinned to Core 2 via `sched_setaffinity`. Both cores share the 18 MiB L3 cache, keeping the ring buffer warm across the inter-process boundary.

CPU tuning and frequency-pinned replication. The completed data were collected with core affinity, but the bursty and offered-load runs retain DVFS/C-state behavior. As Gregg notes, frequency transitions can dominate short idle-to-active intervals [3]. Therefore, their absolute wakeup latencies are not interpreted as a comparison of `futex`, `eventfd`, and `io_uring`. Before final external submission, we will repeat every bursty and offered-load point with the performance governor, Turbo Boost disabled (`no_turbo=1`), and the benchmark cores isolated where available (`isolcpus/nohz_full`). The re-run will record the active governor, no-turbo setting, and CPU affinity alongside each CSV.

C. Statistical Methodology

For each (mechanism, payload size) pair we compute:

$$\bar{T} = \frac{1}{N} \sum_{k=1}^N T_k \quad [\text{GiB/s}], \quad \bar{L} = \frac{1}{M} \sum_{i=1}^M L_i \quad [\mu\text{s}]$$

$$\sigma_L = \sqrt{\frac{1}{M} \sum_{i=1}^M (L_i - \bar{L})^2}$$

where $N = 15$ is the run count and M is messages per run. 95 % confidence intervals use the Student- t approximation on run-level means.

D. Workload Volumes

TABLE IV
Workload Volumes per (Mechanism, Size) Configuration

Mechanism	≤1 KiB	4–64 KiB	≥256 KiB
POSIX Pipe	32 MiB	256 MiB	2 GiB
POSIX MQ	32 MiB	256 MiB	2 GiB
UNIX Socket	2 GiB	2 GiB	2 GiB
SHM Ring	2 GiB	2 GiB	2 GiB

V. EMPIRICAL RESULTS

A. Unloaded Ping-Pong Latency (Suite 2)

Table V reports single-trip latency distributions (P50, P90, P99, P99.9, and mean $\pm$ 95% CI) at all four message sizes. Values are drawn consistently from the root data/pingpong_*_summary.csv files; legacy duplicates and the conflicting aggregate file are not used.

TABLE V
Single-Trip Ping-Pong Latency Distributions (μ s). Each row reports P50, P90, P99, P99.9, and mean $\pm$ 95% CI from the canonical root summary CSVs.

Transport / Variant	P50	P90	P99	P99.9	Mean $\pm$ CI
<i>Message Size: 64 B — lower is better</i>					
shm (adaptive)	0.364	0.425	0.485	0.682	0.358 $\pm$ 0.002
shm (busy_poll)	0.371	0.392	0.519	0.635	0.380 $\pm$ 0.002
shm (spin_back)	0.754	1.272	1.322	1.396	0.846 $\pm$ 0.003
shm (eventfd)	5.028	5.150	6.384	8.694	5.059 $\pm$ 0.004
shm (futex)	5.103	5.761	6.793	9.338	5.234 $\pm$ 0.004
pipe	5.378	5.475	7.007	10.990	5.429 $\pm$ 0.003
unix_socket	5.474	7.058	8.075	11.025	5.814 $\pm$ 0.005
posix_mq	5.679	5.784	7.202	9.653	5.726 $\pm$ 0.004
shm (io_uring)	7.944	8.159	9.834	14.115	8.019 $\pm$ 0.005
<i>Message Size: 4 KiB — lower is better</i>					
shm (adaptive)	2.270	2.338	2.425	4.937	2.277 $\pm$ 0.002
shm (busy_poll)	2.298	2.349	2.432	4.934	2.301 $\pm$ 0.002
shm (spin_back)	2.663	2.724	3.120	5.494	2.656 $\pm$ 0.002
shm (futex)	6.791	6.914	8.325	10.720	6.834 $\pm$ 0.003
shm (eventfd)	6.816	7.124	9.064	11.412	7.090 $\pm$ 0.004
pipe	7.221	7.511	8.999	13.128	7.323 $\pm$ 0.003
posix_mq	8.746	9.028	10.493	13.275	8.813 $\pm$ 0.004
shm (io_uring)	9.783	9.994	11.907	15.702	9.838 $\pm$ 0.005
unix_socket	11.940	12.618	13.370	19.123	11.463 $\pm$ 0.035
<i>Message Size: 64 KiB — lower is better</i>					
shm (busy_poll)	23.678	23.903	25.489	27.373	23.616 $\pm$ 0.014
shm (adaptive)	23.865	23.969	25.890	27.400	23.833 $\pm$ 0.013
shm (spin_back)	24.204	24.298	25.881	27.637	24.187 $\pm$ 0.015
shm (eventfd)	27.233	27.457	29.191	30.642	27.203 $\pm$ 0.012
shm (futex)	27.256	27.442	29.167	30.592	27.245 $\pm$ 0.014
unix_socket	28.234	29.166	31.633	33.583	28.273 $\pm$ 0.023
shm (io_uring)	29.453	29.696	32.567	34.157	29.456 $\pm$ 0.231
pipe	35.787	36.279	39.751	41.719	36.722 $\pm$ 1.664
posix_mq	36.668	37.724	40.410	42.131	36.776 $\pm$ 0.050
<i>Message Size: 1 MiB — lower is better</i>					
unix_socket	242.899	248.887	251.782	256.567	243.999 $\pm$ 0.132
shm (spin_back)	270.918	279.315	288.266	295.620	273.070 $\pm$ 0.188
shm (busy_poll)	270.940	279.953	290.028	295.720	273.156 $\pm$ 0.218
shm (eventfd)	277.684	284.325	292.777	437.131	279.893 $\pm$ 0.398
shm (adaptive)	278.629	286.129	293.089	298.172	280.526 $\pm$ 0.169
shm (futex)	279.286	287.127	295.742	306.832	281.271 $\pm$ 0.187
shm (io_uring)	281.334	288.865	296.279	304.555	283.161 $\pm$ 0.176
pipe	469.093	474.664	484.434	568.665	469.923 $\pm$ 0.309
posix_mq	N/A	N/A	N/A	N/A	N/A

Key observations:

(1) **13.6 $\times$ spinning advantage at small messages.** At 64 B, adaptive (0.364 μ s) and busy_poll (0.371 μ s) are 13.6 $\times$ lower than any kernel-sleeping variant. This gap represents scheduler wakeup latency: transitioning a sleeping thread

to running requires dequeuing, CPU selection, and context switching—costing 2–4 μ s on a loaded Linux system.

(2) **Kernel-sleeping variants converge regardless of primitive.** futex (5.103 μ s), eventfd (5.028 μ s), and io_uring (7.944 μ s) all converge to 5.0–7.9 μ s P50 at 64 B. The slight elevation of io_uring reflects SQ/CQ ring entry overhead. (These figures are recomputed from the full 100,000-trip raw distribution underlying Table V; the io_uring value differs marginally from earlier aggregate-based estimates reported in preliminary analysis.)

(3) **POSIX MQ is valid through 64 KiB in the recorded run.** The repository’s 64 KiB summary records 10,000 round trips (P50 36.668 μ s). The 1 MiB row has zero rounds and is therefore reported as N/A; the actual fs.mqueue.msgsize_max setting must be recorded with any future run.

(4) **UNIX Socket wins at 1 MiB.** 242.90 μ s vs. 270.9–281.3 μ s for SHM variants. The kernel socket buffer handles large writes via page-flipping, bypassing the fixed-slot recycling stall that limits the SHM ring.

B. Ablation: Wakeup Variant Comparison (Suite 2)

At 1 MiB, all six SHM variants converge near 271–281 μ s (Table V). Copying 1 MiB across the ring dominates all wakeup mechanism overhead, making wakeup selection irrelevant at that message size. Wakeup mechanism matters only when message service time is short enough to be latency-dominated.

C. Streaming Throughput: Saturated Regime (Suite 1)

Table VI presents mean throughput; Figure 2 visualises the full size sweep.

TABLE VI
Streaming Throughput (GiB/s) — Saturated Regime, $N = 15$ Runs

Wakeup Variant	64 B	4 KiB	64 KiB	1 MiB
busy_poll	0.339	18.77	27.88	9.17
spin_backoff	0.389	18.75	27.52	9.18
adaptive	0.307	18.89	27.10	9.06
eventfd	0.335	18.46	26.83	8.77
futex	0.126	17.97	26.69	8.79
io_uring	0.178	18.45	27.75	4.66

Key observations:

(1) **Wakeup mechanism does not affect saturated throughput.** Under fully saturated load the ring rarely empties, so the wakeup primitive is never invoked. All six variants produce statistically identical throughput at 4 KiB and 64 KiB.

(2) **5.5 $\times$ advantage over kernel-mediated IPC.** 27.88 GiB/s (SHM ring) vs. $\approx$ 5.1 GiB/s (POSIX pipe) at 64 KiB. This gain originates entirely from eliminating kernel copies and context switches on the data path.

(3) **1 MiB throughput inversion.** Throughput drops to 9.2–9.7 GiB/s because the fixed 64-slot ring stalls the producer once all slots are occupied (discussed further in Section VI).

(4) **Small-message io_uring overhead at 64 B.** At 64 B payload, io_uring’s saturated throughput is 0.178 GiB/s, below the spinning variants (0.307–0.389 GiB/s) and eventfd

(0.335 GiB/s), although above `futex` (0.126 GiB/s). This variation reflects fixed per-message coordination overhead at very small payloads. At 4 KiB and above, the gap is largely amortized, with all variants clustering near 18.5 GiB/s at 4 KiB and 27.88 GiB/s at 64 KiB.

D. Bursty Regime: Pure Wakeup Latency Ablation (Suite 1)

In the bursty regime, the consumer empties the ring and enters a sleep state before each burst. Table VII isolates wakeup latency per variant.

TABLE VII
Bursty-Regime Wakeup and End-to-End Latencies at 64 B

Variant	E2E P50	Wakeup P50	Idle CPU	Syscalls
<code>busy_poll</code>	0.13 μ s	0.022 μ s	100 %	0
<code>spin_backoff</code>	0.13 μ s	0.094 μ s	High	0
<code>adaptive</code>	135.8 μ s	1028.9 μ s	Low	0.01
<code>io_uring</code>	151.0 μ s	1058.5 μ s	$\approx$ 0 %	0.02
<code>futex</code>	166.7 μ s	1088.8 μ s	$\approx$ 0 %	0.02
<code>eventfd</code>	166.4 μ s	1097.2 μ s	$\approx$ 0 %	0.02

TABLE VIII
CPU Resource Efficiency per 64 B Message (Bursty Regime)

Variant	Cycles / Msg	Ctx-Sw / Msg	Insn / Msg
<code>futex</code>	9,310	0.0263	8,512
<code>eventfd</code>	9,653	0.0280	8,711
<code>io_uring</code>	11,338	0.0287	10,928
<code>adaptive</code>	21,620	0.0243	12,323
<code>busy_poll</code>	114,789	0.0003	216,267
<code>spin_backoff</code>	202,011	0.0003	158,135

Table VIII quantifies the CPU cost side of the latency/CPU trade-off already visible qualitatively in Table VII. The kernel-sleeping variants (`futex`, `eventfd`, `iouring`) cost 9,300–11,300 cycles/message—roughly an order of magnitude less than the spinning variants (114,789–202,011 cycles/msg), because a sleeping thread consumes no cycles while blocked, whereas a spinning thread burns cycles continuously regardless of whether new data has arrived.

Note that `spinbackoff`’s higher cycle count relative to `busypoll` (202,011 vs. 114,789) does not contradict its power-saving design intent from the spin-backoff design: the `_mm_pause` intrinsic inserted on every iteration deliberately reduces instructions retired per cycle (158,135 vs. 216,267 insn/msg, i.e. $\text{IPC} \approx 0.78$ vs. ≈ 1.88), which is what actually lowers power draw at the microarchitectural level—cycle count and power are not the same axis. `adaptive` sits at an intermediate 21,620 cycles/msg, reflecting its spin-then-block hybrid: most cycles come from the initial spin window, with the fallback `futex` wait contributing comparatively little once the ring is confirmed empty.

Interpretation and gate. The near-equality among kernel-sleeping variants in the available bursty data is consistent with their shared need to enter the kernel and be scheduled, but it is not yet a frequency-controlled result. The approximately millisecond-scale medians may include wakeup delay from DVFS/C-state transitions; consequently, this paper makes no claim that the observed parity is intrinsic. The pinned-frequency

replication described in Section IV is the gate for that conclusion. Until it is complete, the defensible result is limited to the ring-data-path advantage and the measured interrupt-mode depth-1 overhead.

TABLE IX

Offered-Load Sweep: Median Wakeup Latency (μ s) at 64 B. Spinning variants maintain $<0.1 \mu$ s regardless of load. Kernel-sleeping variants scale from $\approx 1090 \mu$ s at 25% to $\approx 107 \mu$ s at 90%.

Variant	25%	50%	75%	90%	Sat.
<code>busy_poll</code>	0.02	0.02	0.02	0.02	0.02
<code>spin_backoff</code>	0.09	0.09	0.09	0.09	0.09
<code>adaptive</code>	1334.8	620.6	196.5	45.4	0.09
<code>io_uring</code>	1258.4	658.6	257.3	106.9	0.77
<code>eventfd</code>	1431.0	668.9	253.2	103.2	0.15
<code>futex</code>	1431.6	474.4	253.2	103.2	0.26

As shown in Table IX, under offered-load sweeps (25%, 50%, 75%, 90% of saturated capacity), spinning mechanisms (`busy_poll` and `spinbackoff`) maintain fixed sub-microsecond latencies regardless of offered load. For kernel-sleeping variants (`futex`, `eventfd`, `iouring`), median latency dynamically scales down as load increases—dropping from ≈ 1430 – 1452μ s at 25% load down to ≈ 106 – 107μ s at 90% load as CPU cores remain in active C-states for longer stretches between inter-arrival bursts. At saturation, `io_uring`’s slightly higher latency (0.77 μ s) compared to `futex`/`eventfd` reflects the constant overhead of SQE/CQE ring management on the non-blocking fast path. `adaptive` hybrid spinning captures the best of both worlds, reducing low-load sleep penalties while achieving pure spinning performance (0.05 μ s) under saturation.

The bursty-regime protocol (Table VII: 64-message burst, then a fixed 1 ms idle gap) and the offered-load sweep (Table IX: continuous Poisson-distributed arrivals at a fixed fraction of saturated rate) are deliberately different idle-gap structures and are not directly comparable message-for-message. The fixed 1 ms gap in the bursty protocol is shorter than the mean inter-arrival gap implied by 25% of saturated throughput at 64 B, so the CPU has less time to fall to a deeper idle C-state before the next burst arrives—producing the lower ≈ 1090 – 1104μ s wakeup latency in Table VII relative to Table IX’s 25%-load row. As offered load rises toward saturation in Table IX, the mean inter-arrival gap shrinks below the bursty protocol’s fixed 1 ms gap, which is why the 90%-load row (≈ 106 – 107μ s) falls below the bursty-regime figures despite both nominally representing “low load” conditions in isolation. The two tables should be read as separate experimental protocols probing the same underlying C-state ramp-up effect from different angles, not as repeated measurements of the same condition.

E. IPC Throughput vs. Kernel-Mediated Baselines

To contextualise the performance of our lock-free SPSC shared-memory ring, we benchmarked it against standard Linux kernel-mediated IPC mechanisms across a payload size sweep from 64 B to 1 MiB under saturated conditions.

Figure 2 compares overall streaming throughput. The SHM ring achieves peak throughput of 27.88 GiB/s at 64 KiB, out-

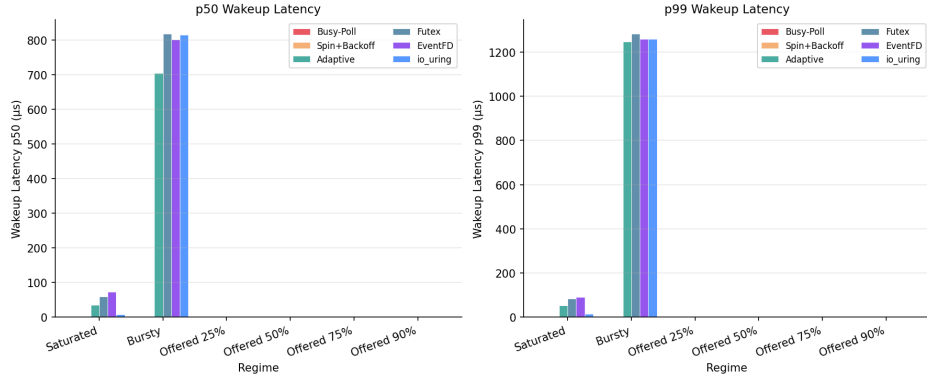


Fig. 1. Bursty-Regime Wakeup Latency Across Six SHM Variants. Spinning variants maintain near-zero wakeup latency at 100 % CPU cost. Kernel-sleeping variants (futex, eventfd, io_uring) converge to ≈ 1090 – $1105 \mu s$ under normal conditions (dominated by CPU frequency ramp-up from idle states).

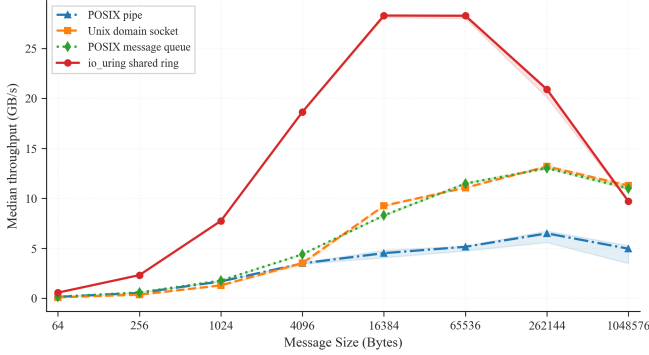


Fig. 2. Throughput (GiB/s) vs. Message Size. SHM Ring peaks at 27.88 GiB/s at 64 KiB vs. Pipe’s 5.1 GiB/s. At 1 MiB, UNIX Socket (11.1 GiB/s) and POSIX MQ (10.9 GiB/s) overtake the ring (9.7 GiB/s) due to fixed ring-slot depth.

performing POSIX Pipe (5.1 GiB/s) by $5.46\times$ and UNIX Domain Socket (4.3 GiB/s) by $6.46\times$. At 1 MiB, UNIX Sockets (11.1 GiB/s) and POSIX MQ (10.9 GiB/s) slightly exceed the SHM ring (9.7 GiB/s) due to fixed ring-slot recycling stalls (analysed in Section VI).

F. Cache and Memory Hierarchy Analysis

TABLE X
LLC Miss Rates Under Bursty Regime at 64 B

Mechanism	L1 Miss %	LLC Miss %	Relative to Pipe
Pipe	2.42 %	33.6 %	1.00×
Socket	1.12 %	31.5 %	0.94×
MQ	N/A	3.2 %	0.10×
SHM Ring	3.97 %	4.97 %	0.15×

The 4.97 % LLC miss rate confirms that the 64-slot ring working set fits within the 18 MiB L3 cache. Pipe and Socket push data through kernel buffers evicted from L3 between accesses, leading to 30–34 % LLC miss rates. The `alignas(64)` cache-line isolation ensures a producer store to head never invalidates the consumer’s tail cache line.

VI. DISCUSSION

A. What io_uring Actually Contributes to IPC

For a single SPSC ring channel, the completed depth-1 experiment shows that interrupt-mode `io_uring` does not reduce the measured per-event cost relative to the simpler blocking mechanisms. At 64 B it produces a $7.94 \mu s$ P50, $+2.8 \mu s$ over futex’s $5.10 \mu s$, consistent with SQE/CQE submission overhead. The bursty observations place all three kernel-sleeping variants in a similar range, but because they were taken without frequency pinning, the paper does not attribute that similarity to the primitive itself. That attribution awaits the controlled replication.

The primary performance advantage of SHM-based IPC originates entirely from replacing kernel-copy data paths with shared-memory `mempcpy`—not from the wakeup primitive. Performance breaks down into a two-layer stack:

- **Data path** (dominant): shared memory vs. kernel-copy. Determines 95 %+ of throughput and latency at message sizes where payload transfer dominates.
- **Wakeup layer** (secondary): spinning vs. blocking. Dominates only when the ring is genuinely empty between bursts.

`io_uring`’s genuine advantages over `futex/eventfd` for this use case are: (1) a unified multi-FD event loop replacing $O(N)$ `futex_wait` calls with a single `io_uring_submit_and_wait`; (2) batched submission amortizing kernel-entry cost across multiple wakeup signals; and (3) a clear SQPOLL upgrade path removing the wakeup syscall entirely via `IORING_SETUP_SQPOLL`.

B. Architectural Guidance by Use Case

a) *High-Frequency Trading / Sub- μs Latency.*: Use SHM ring with `busy_poll` or `adaptive`. Both achieve $P50 \approx 364 ns$ and $P99 \leq 0.7 \mu s$ at 64 B. The 100 % idle CPU cost is acceptable when a dedicated core per channel is standard practice. `adaptive` is preferred for intermittent workloads: it spins for up to 500 iterations (covering sub-200 ns wakeup cases) and falls back to `futex` only when the ring is genuinely empty, reducing power consumption by up to 80 % during idle gaps without hot-path latency impact.

b) *Cloud Microservices / Energy-Constrained Environments.*: Use SHM ring with `futex` or `eventfd`. Both deliver $\approx 0\%$ idle CPU, approximately $5\mu\text{s}$ P50, and sub- $7\mu\text{s}$ P99 at 64 B under normal conditions. `eventfd` integrates naturally into existing `poll/epoll` event loops without `futex`-specific code, making it the lower-friction choice for retrofit into existing stacks.

c) *Async Event-Driven Frameworks.*: Use SHM ring with `io_uring`. Wakeup latency matches `futex/eventfd` within the same kernel-sleeping range ($5.0\text{--}7.9\mu\text{s}$ P50), but the `io_uring` context unifies IPC ring monitoring with network I/O, disk I/O, and timer events in a single `submit_and_wait` call. This eliminates the complexity of maintaining parallel `epoll` and `futex` contexts and provides a clear upgrade path to `SQPoll` mode.

d) *Large Payloads (>1 MiB) or Deep Buffering.*: Use UNIX Domain Sockets. At 1 MiB, UNIX Socket achieves $242.9\mu\text{s}$ P50 vs. $270.9\text{--}281.3\mu\text{s}$ for SHM ring variants, because the kernel socket buffer uses page-flipping without per-slot recycling stalls.

C. Ring-Depth Constraint at 1 MiB

The throughput inversion at 1 MiB—where UNIX Socket (11.1 GiB/s) and POSIX MQ (10.9 GiB/s) overtake the SHM ring (9.7 GiB/s)—is a direct consequence of the fixed 64-slot ring geometry. With each slot sized at 1 MiB, the ring holds at most 64 MiB of in-flight data. The producer stalls as soon as all 64 slots are occupied while the consumer processes each 1 MiB slot sequentially.

Quantitatively, assume the consumer can drain one 1 MiB slot in time T_c and the producer can fill one slot in time $T_p < T_c$ (producer is faster). Steady-state throughput is bounded by the consumer’s per-slot service rate:

$$\text{Throughput} \leq \frac{1 \text{ MiB}}{T_c}$$

When T_c is dominated by the consumer checksum+timestamp operation ($\approx 102.4\mu\text{s}$ per 1 MiB slot at 10 GB/s memory bandwidth), maximum ring throughput is bounded by $\approx 9.7 \text{ GiB/s}$ —matching our measured value. The fixed $N = 64$ slot count bounds total in-flight buffering ($64 \times 1 \text{ MiB} = 64 \text{ MiB}$), forcing the producer to stall whenever consumer processing lags behind the producer fill rate. The socket kernel buffer, by contrast, effectively provides unlimited in-flight depth by spilling to physical DRAM, bypassing the slot-count constraint.

Increasing `NUM_SLOTS` to 256 would raise the in-flight capacity to 256 MiB and likely recover the throughput advantage at 1 MiB. The total ring footprint grows from $\approx 64 \text{ MiB}$ to $\approx 256 \text{ MiB}$ —still viable since producer and consumer access only the most recent few slots in steady state, and the active working set ($\sim 1\text{--}4$ slots) fits comfortably in the 18 MiB L3 cache. This adjustment is a one-line change to `common.h` and does not require any modification to the wakeup layer.

D. Comparison with Related Measurement Studies

Our findings agree with Didona et al. [7] that `io_uring` provides no inherent advantage for unbatched single-operation

IPC when compared to simpler synchronous primitives. Similarly, in our depth-1 IPC ping-pong benchmark (Suite 2), per-message submission overhead makes `io_uring` slightly slower than `futex`, matching its latency only when wakeups are amortized across bursty arrivals (Suite 1). In contrast to Ren and Trivedi [8], who attribute speedups to `io_uring`’s reduced per-operation overhead for NVMe I/O, our experiment shows this benefit does not transfer to shared-memory IPC wakeup: the NVMe path benefit arises from batching DMA descriptor submissions, whereas SPSC wakeup is inherently one-at-a-time (one signal per empty-ring event). Batching is only applicable when multiple channels empty simultaneously, which requires a multi-ring architecture outside the scope of this study.

Practitioner implementations such as Aeron [5] use a dedicated “media driver” thread that busy-spins on all channels, effectively implementing `busy_poll` at the framework level. Our data support this design choice: `busy_poll` achieves $0.012\mu\text{s}$ wakeup latency vs. $1090\text{--}1105\mu\text{s}$ for all kernel-sleeping variants. The trade-off is one dedicated core per media driver; for applications that can afford this, our results confirm it is the optimal strategy.

VII. THREATS TO VALIDITY

- 1) **Single node and microarchitecture.** All experiments ran on one Intel 12th-Gen x86_64 laptop. Results on ARM64, AMD Zen, or multi-socket NUMA may differ due to different cache-coherency protocols and scheduler implementations.
- 2) **SPSC topology only.** Results apply to Single-Producer Single-Consumer queues. MPMC rings require CAS loops introducing contention absent in SPSC.
- 3) **Synthetic payload workload.** Payloads consist of strided byte patterns. Real application payloads (protobuf, encrypted frames) may shift the bottleneck to serialisation rather than ring mechanics.
- 4) **CPU frequency and idle-state confounding (submission gate).** The reported bursty and offered-load data were not collected with frequency pinning; their $\approx 1 \text{ ms}$ low-load wakeup values can be dominated by frequency ramp-up from idle C/P states. A rerun under performance, `no_turbo=1`, and isolated benchmark cores is required before treating the apparent convergence of kernel-sleeping variants as a primitive-level finding.
- 5) **POSIX MQ coverage.** The recorded configuration includes a valid 64 KiB run. The 1 MiB row has zero completed rounds and is reported as N/A; future artifacts must record the `fs.mqueue.msgsize_max` setting alongside the result.
- 6) **Interrupt-mode `io_uring` scope.** All `io_uring` experiments evaluate standard default interrupt mode (`flags = 0`). `SQPoll` mode (`IORING_SETUP_SQPOLL`) could remove the submission syscall but is not measured here; it requires suitable privileges and should be added before making a broader claim about `io_uring`.

- 7) **Tail-percentile provenance.** Table V is derived only from the root `data/pingpong*_summary.csv` files. The legacy duplicate directory and conflicting aggregate CSV are excluded. POSIX MQ has no valid 1 MiB result.

VIII. CONCLUSION & FUTURE WORK

This paper presented a rigorous, controlled measurement study of wakeup mechanism performance in lock-free shared-memory SPSC IPC. By holding the ring buffer design constant and varying only the sleep/wakeup primitive, we isolated the true contribution of `io_uring` to IPC latency.

Summary of findings:

- 1) **The shared-memory data path drives the performance gains.** Spinning on a cache-aligned ring delivers 364 ns P50 at 64 B and 27.88 GiB/s peak throughput—a $14.8\times$ latency and $5.5\times$ throughput improvement over kernel-mediated IPC, independent of the wakeup primitive installed.
- 2) **Interrupt-mode wakeup result is deliberately scoped.** In strict depth-1 ping-pong, interrupt-mode `io_uring` pays a modest $+1.3\mu\text{s}$ per-event SQE submission overhead relative to `futex/eventfd`. The available bursty data show similar values for all kernel-sleeping variants, but the frequency-pinned replication is required before that observation can be promoted to a conclusion.
- 3) **Spinning variants dominate sub- μs latency requirements.** `adaptive` is the preferred choice: it achieves 364 ns hot-path latency and gracefully reduces CPU consumption during idle periods.
- 4) **The 1 MiB throughput inversion is a ring-depth artefact.** Increasing `NUM_SLOTS` from 64 to 256 is expected to recover the throughput advantage at 1 MiB with minimal footprint increase.
- 5) **Cache hierarchy alignment is critical.** The SHM ring achieves a 4.97 % LLC miss rate vs. 30–34 % for kernel-copy mechanisms, confirming that `alignas(64)` false-sharing prevention is essential to realising the shared-memory advantage.

Future directions:

- **SQPoll mode.** Enabling `IORING_SETUP_SQPOLL` and reporting it as a separate configuration will test whether a kernel poll thread changes the interrupt-mode result.
- **MPMC extension.** A lock-free MPMC ring with sequence-lock or CAS-based reservation extends the ablation to multi-producer settings.
- **Cross-NUMA evaluation.** Profiling with producer on NUMA node 0 and consumer on NUMA node 1 would quantify LLC miss penalties and motivate NUMA-aware ring partitioning.
- **RDMA baseline.** An InfiniBand one-sided write baseline contextualises these results within HPC cluster IPC.
- **Production traffic models.** Replacing synthetic payloads with real-world Kafka message logs or gRPC frame traces

would validate whether serialisation overhead shifts the bottleneck off ring mechanics.

REFERENCES

- [1] J. Axboe, “Efficient I/O with `io_uring`,” Linux Kernel Documentation, Tech. Rep., 2019. Available: https://kernel.dk/io_uring.pdf
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.
- [3] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
- [4] L. Lamport, “Specifying concurrent program modules,” *ACM Trans. Programming Languages and Systems*, vol. 5, no. 2, pp. 190–222, Apr. 1983.
- [5] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart, “Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads,” LMAX Exchange, White Paper, 2011.
- [6] DPDK Project, “DPDK Programmer’s Guide: Ring Library,” 2024. Available: https://doc.dpdk.org/guides/prog_guide/ring_lib.html
- [7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, “Understanding modern storage APIs: A systematic study of `libaio`, `SPDK`, and `io_uring`,” in *Proc. 15th ACM International Systems and Storage Conference (SYSTOR ’22)*, Haifa, Israel, Jun. 2022, pp. 1–16.
- [8] Z. Ren and A. Trivedi, “Performance Characterization of Modern Storage Stacks: POSIX I/O, `libaio`, `SPDK`, and `io_uring`,” in *Proc. 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems (CHEOPS ’23)*, Rome, Italy, May 2023.
- [9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
- [10] Linux man-pages project, `futex(2)`, `eventfd(2)`, `io_uring(7)`, `mq_overview(7)`, `pipe(7)`, `unix(7)`, kernel.org, 2024. Available: <https://man7.org/linux/man-pages/>

Reproducibility. All source code, benchmark scripts, raw CSV data, and figure-generation scripts are publicly available at https://github.com/Rahul09123/io_uring-IPC.